

CAPL-2





Agenda

During the last lesson

- Introduction of CAPL in CANoe
- CAPL syntax and semantics
- CAPL events and event procedures

During this lesson

- CAPL symbol access
- CAPL and panels
- CAPL functions







Symbolic access

- Signal values are generally accessed as physical values, regardless of the scaling of message transmission. This is set in the database and is taken from there.

> Physical access to signal values:

```
// Definition of the representation in the database  
$EnergyMgmt::BatteryVoltage = 14.1;
```

> Access to raw value of a signal:

```
// 8 to 18 Volt with 12bit resolution, without range check  
$EnergyMgmt::BatteryVoltage.raw = (14.1 - 8) / (18 - 8) * 4096;
```

> Access on a message base:

```
// Most significant bytes Motorola, of 12 bits only the lower 4 bits are used  
msg.byte(0) = (msg.byte(0) & 0xF0) | (byte)((14.1 - 8) / (18 - 8) * 4096 / 256) & 0xF;  
// Least significant byte  
msg.byte(1) = (byte)((14.1 - 8) / (18 - 8) * 4096) & 0xFF;  
output(msg);
```



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Symbolic access

- The database basically follows the architecture on the right.

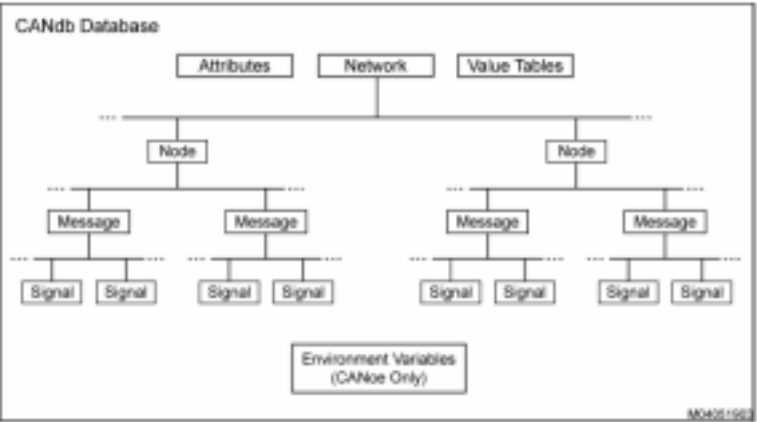
○ **Attribute** is a distinct characteristic of an object in the database (network, node, message, signal, environment variable etc.). Most of the time, attributes are used to represent a relationship between each object of the same type (an attribute for cycle time)

○ **Value tables** are used to define symbolic values. Some

data actually makes more sense if it is displayed symbolically rather than with their real value (hex, dec, eng units).

- **Network node** consists of one CAN Controller and a transceiver in an ECU. An ECU may contain more than one node. A node may have a ID and it is responsible for a set of functions of that ECU.
- **Message** is a container holding a block of data transmitted into the bus. This block of data is shown numerically in CANoe in either hex or dec. To display the message name and its data field, in engineering units or symbolically, the message has to be defined in the database.

Database Objects	
Object Type	Description
Networks	A CAN bus network that contains multiple ECUs to exchange data
ECUs	Distributed processing units within a network (for example, Traction Control, ABS, Adaptive Cruise Control)
Nodes	Network interfaces of an ECU used to transmit and receive data from the CAN bus
Messages	Containers of information that are transmitted on the CAN bus
Signals	Independent data regions in the data field of a message
Environment Variables	Input and output variables of network nodes, such as switch position, sensor signals and actuator signals





Symbolic access

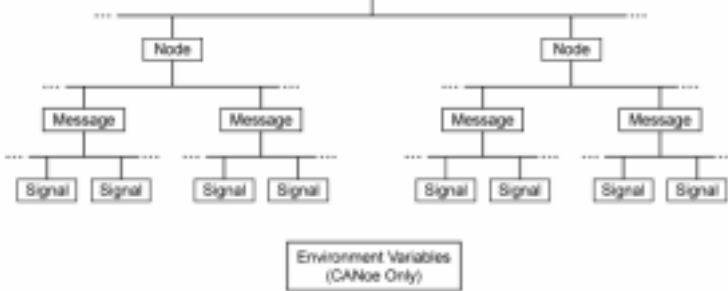
- The database basically follows the architecture on the right.
- **Signals** are the actual data objects to be exchanged between nodes, and are encoded in the data field of CAN messages. Signals typically occupy most or all of a CAN message data bytes, and they must not overlap. In the database, signals can have parameters for conversion of raw data to physical units (also known as an engineering unit).
- **Environment variables** are data objects global to the CANoe environment, and are used to link the functions of a CANoe panel to CAPL programs.

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



CANdb Database

Attributes Network Value Tables



Model 1503

cineton





PANEL

- In CANoe: Home -> Panel -> New Panel
- Communication between panels and CAPL involves environment variables.
- All dynamic control elements within a panel should be associated with an environment variable while the panel is being created.
- When the user clicks or adjusts the value of a control, the associated environment variable value will change as an environment variable event procedure is executed in CAPL.
- Env variables should already be defined in the database, along with messages and signals. If they are defined in the database, CAPL will have access to them.
- The value of the environment variables can also be changed by CAPL. These changes will also affect the displayed controls on the panel

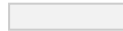
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



- In this example, the analog gauge:
 - mV (default unit), it can be changed in the unit you need by editing Text mV in the *Properties*.
 - Layout Style 90, it can be changed with the angle which better suite the time of gauge you want to use.
 - Maximum 100 (default), you can change it with the maximum allowed value in the assigned signal from dbc.
 - The same with Minimum 0 (default).

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Workable are



PANEL

- However, you can attach a signal to a panel element to show its value during the simulation.





PANEL

- Panels are not used to exchange data between simulated network nodes. That is, environment variables are not capable of exchanging data between a simulated node and a real module. The only way to exchange data in a CAN network is by using a CAN message.

- Environment variables are global and may be used in every network node in a CANoe configuration. Use them to exchange data is a mistake, the results lurking unknown until one or more real modules are connected to CANoe to communicate with the simulated network nodes.
- Environment variables have to be assigned a data type in the database. The choice of data type includes INTEGER, FLOAT, STRING, and DATA. STRING is used for ASCII. DATA is used to store values in term of bytes.





Functions

- By default, CANoe initializes all environment variables to their default value assigned in the database

whenever the measurement starts.

- Sometimes it is necessary to initialize all environment variables at the beginning of the measurement before their values change during the measurement, even if these initial values are different than the default values in the database. CAPL provides the **callAllOnEnvVar()** function to execute all environment variable event procedures.
- You would normally call **callAllOnEnvVar()** in **on start** event procedure to initialize your environment variables. Related actions may include the following:
 - Initializing variables
 - Setting timers to activate in response to changes in environment variables
 - Sending out messages onto the CAN bus containing the starting values of your environment variables.
- **N.B.** Environment variables can only be defined in a database, so they cannot be declared in the CAPL language.

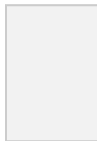




Functions

- The value of an environment variable can only be changed by two methods. The most common method is through a panel. The second way is by calling the **putvalue()** function.
- The **putValue()** function is used to set or initialize an environment variable. This function takes two or three parameters, depending on the data type of the environment variable.
- N.B. Initialize means that this function cannot be used to reset env variable value during the running of our program (after on start)
- **Note:** Do **not** use **this** keyword with the **putValue()** function in an env. Variable event procedure, because that may cause the event to execute in an infinite loop.





Functions

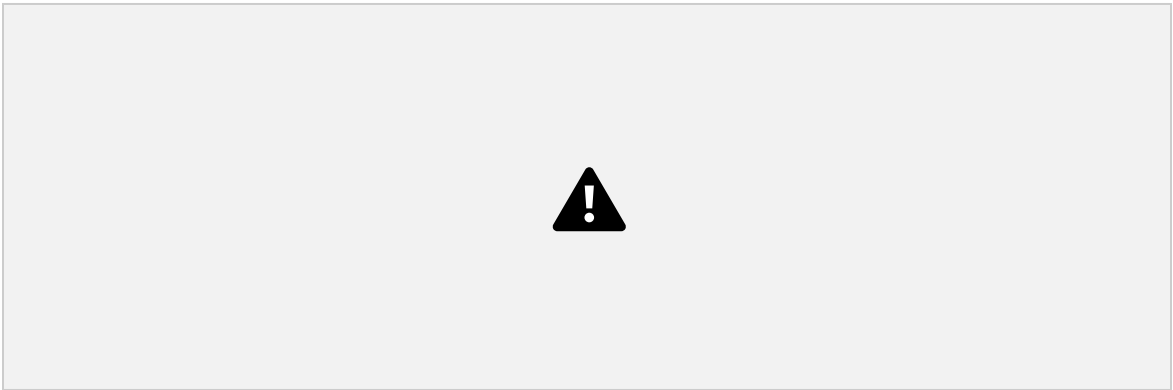




Functions

- Calling the **getValue()** function will return the environment variable value. It has five formats like the **putvalue()** function.
- The formats return the active value of the environment variable for integers and floats. If an array is required as in the STRING and DATA data-type, the function returns the number of bytes copied.
- **this** keyword is used in conjunction with the **getValue()** function to access the value of the environment variable. an example

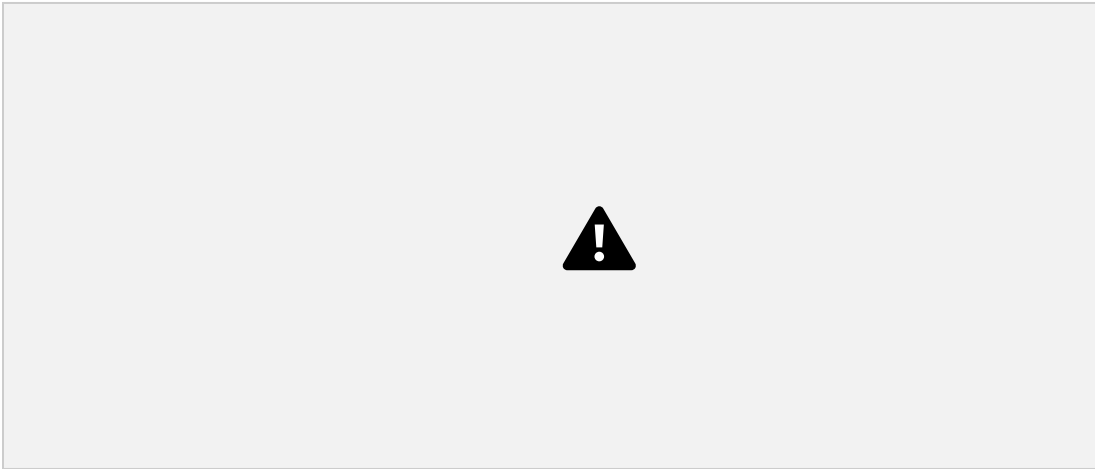




Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Functions





this document are property of Kineton s.r.l. 2018

Strictly confidential. All contents in

Functions

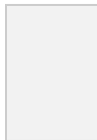
- These environment variables are accessed by three functions: **getValue()**, **getValueSize()**, and **putValue()**.
- The **putValueToControl()** function is used to assign a

value to the multi display control element on a panel without using an environment variable.

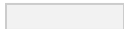
- If a particular object or element in a panel needs to be enabled or disabled, use the **enableControl()** function. It has access to panel help elements, panel recorder elements, panel control buttons, and other elements with association to either an environment variable or signal.
- The elements colour can be set using the **setControlBackColor()**, **setControlForeColor()**, and **makeRGB()** functions.

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018





Functions





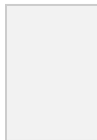
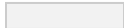
Functions





Strictly confidential. All contents in this document

are property of Kineton s.r.l. 2018



Functions

- When a message is received from the CAN bus, the user may use the **isExtId()** and **isStdId()** functions to check to see if the message has an 11-bit or 29-bit identifier.
- If a message has an 11-bit identifier but a 29-bit is desired, use the **mkExtId()** function to convert it.
- Whenever a message needs to be sent from a node, the **output()** function.



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

- Most of the time, the next functions are used to convert data bytes from Intel format (Little-Endian) to Motorola format (Big-Endian) or vice versa to get the correct signal value.
- If a database is used, the format parameter has already been set up and CANalyzer or CANoe will automatically perform the format changes based on the database.



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Functions

- The **setBtr()** function allows you to set or reset the baud rate for a channel. This function is optional before you reset the CAN controller. If the baud rate has to be changed, this function must be called before the **resetCanEx()** function. If you want to reset all the CAN controllers at once instead of one at a time, use the **resetCan()** function.
- **Note:** Resetting a CAN channel requires the CAN controller to be disconnected. Because of this disconnection, data in both the transmit and receive queues are lost.
- For CANoe, the user has full control. In most situations, the user restarts the CANoe measurement to reset the CAN Controllers. If you do not want the measurement to be restarted, implement the bus off event procedure to have the CAN controller(s) reset itself. Code below:



- CAPL allows you log portions of a simulation. When the conditions to begin logging are met, call the **startLogging()** function in the CAPL code.
- By using CAPL to control logging, all the time settings in the configuration of the Logging block can be offset by the settings that were set with a CAPL program. The pretrigger time is the period of time during which bus activity is recorded before logging is triggered. The posttrigger time is the period of time during which bus activity is recorded after logging is stopped.
- In the following example, pressing key '1' starts logging with a 1000 ms pre-trigger time, and key '2' is used to stop logging with a 2000 ms post-trigger time.



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



- Mathematical functions are used for complex calculations. This is a group of functions that also exist in the C language. Below is an example of how mathematical functions are used:

```
double x;
```

```
x = cos(PI); // returns -1, PI is a built-in constant
```

```
double tangent(double x) // user-defined function tangent()
```

```
{
```

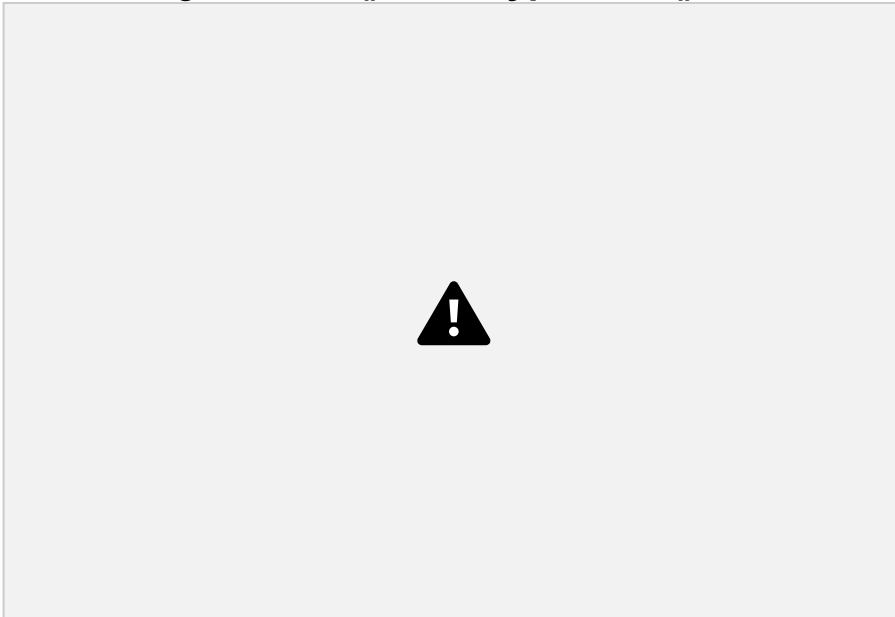
```
    return sin(x) / cos(x);
```

```
}
```

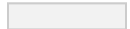


- The most commonly system interface function is the **write()** function. This function is mainly used for debugging by outputting text and values to the Write window of CANalyzer or CANoe.
- an example of sending a message when a key is pressed using the **write()** and **keypressed()** functions:

```
variables
{
  msTimer mytimer; // timer
  message 100 msg; // CAN message
}
on key F1
{
  setTimer(mytimer, 100); // start 100 ms timer
  write("F1 pressed"); // output to Write window
}
on timer mytimer
{
  if (keypressed()) // true if any key is pressed
  {
    setTimer(mytimer, 100); // restart timer
    output(msg); // send message while key is pressed
  }
  else write("F1 released");
}
```

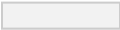


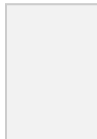
- In CAPL, programmers also need to know how the system clock operates. When a measurement starts, the system clock initializes. It works independently of the Window timers.
- To get the current system time, the **timeNow()** function is used. Every message, sent or received, will also have a system time stamp set by the CAN controller. This time stamp becomes important when you are performing network tests (node responsiveness, for example).
- To calculate the time difference between two messages, the **timeDiff()** function is used, or using the TIME selector.
- If you need to reference the Window clock, the **getLocalTime()** and **getLocalTimeString()** functions are available.
- For controlling processing time variation, CAPL provides two functions: **setDrift()** and **setJitter()**. The CAPL **setDrift()** function sets a constant deviation between –100% and +100% for all timers used in a network node. The **setJitter()** function, however, sets the fluctuation interval between –100% and +100%. Setting either function will reset the other. To set both the drift and the jitter, use the **setJitter()** function only.
- To recall the drift or jitter values, use the **getDrift()**, **getJitterMax()**, and **getJitterMin()** functions.



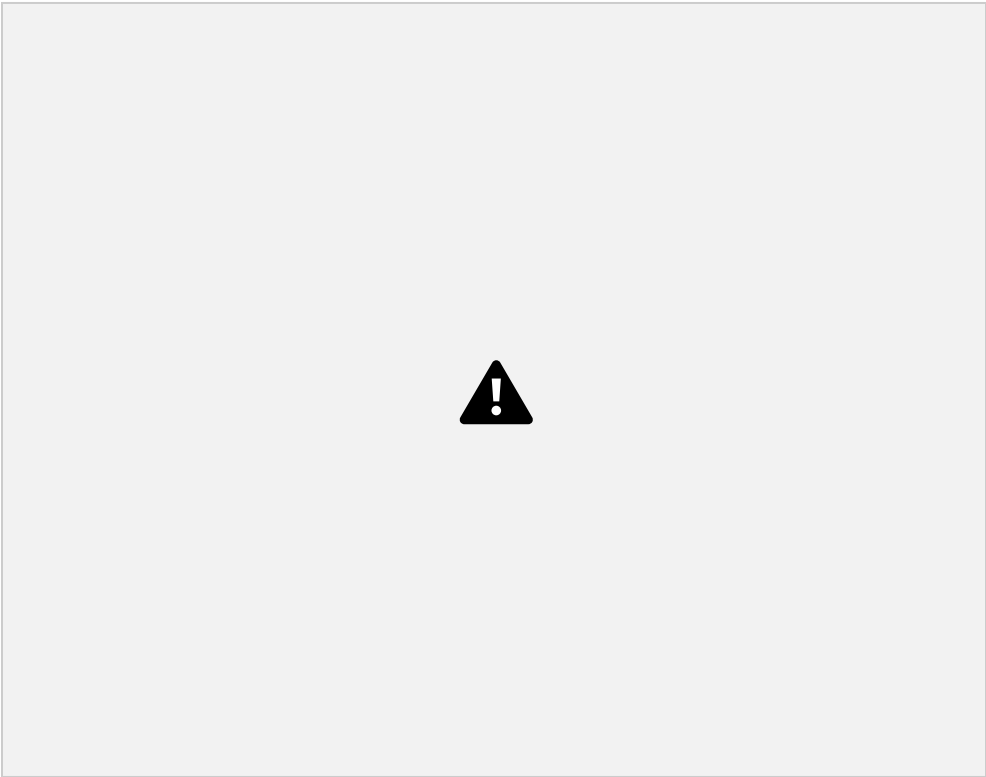
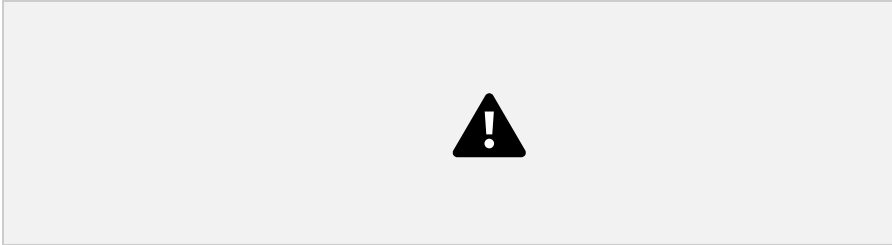


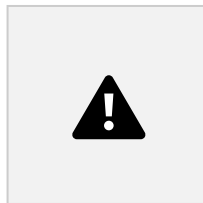
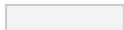
Functions





Functions





Maria Pia Vaia
maria.vaia@kineton.it

Via E. Gianturco, 23
(presso complesso
Mecfond) 80146 Napoli

www.kineton.it